Bellman-Smoothed Regression for Value Function Estimation

Anonymous Authors[1]

Abstract

Value function estimation is a core problem in Reinforcement Learning. Monte-Carlo methods compute an unbiased estimate of the value function, but can suffer variance related to the length and noisiness of the task. Temporal differences methods (TD Learning) are popular in part because they reduce variance, and in part because they induce local Bellman-consistency of the learned value function. However, TD learning can fail to converge when using non-linear function approximation. We study a new learning rule based on the minimization of a convex Bellman-smoothed value loss. We extend previous results that show when TD learning succeeds at lowering the Bellman-smoothed value loss. Our main contribution is using this loss for value function learning in deep RL. We first show how to estimate the new objective in an unbiased fashion from trajectory data – with strictly lower variance than Monte Carlo. We show that in expectation, the learned value function leads to stronger policy improvement than classical TD learning. We experiment on RL tasks, including continuous control, showing that our value function loss improves the performance of PPO.

1. Introduction

2. Preliminaries

$$D \doteq \text{diag}(\boldsymbol{\mu}) \tag{1}$$
$$A \doteq D(I - \gamma P) \tag{2}$$

If the Bellman Operator is a contraction in the D-weighted norm, then A is positive definite (**?**). We make use of the labels S and K for its symmetric and skew symmetric components respectively.

$$A = \frac{1}{2}(A + A^\intercal) + \frac{1}{2}(A - A^\intercal) \tag{3}$$
$$= S + K \tag{4}$$

The objective is to train a differentiable value estimate v_θ, with $|\theta| = k$. The estimated value vector $\boldsymbol{v}$ is defined by stacking each state's value prediction, so that $\boldsymbol{v}(i) = v_\theta(s_i)$. The value error vector is given by

$$\boldsymbol{e} \doteq (\boldsymbol{V} - \boldsymbol{v}). \tag{5}$$

The fundamental goal of policy evaluation is to improve the policy. The instrumental goal is to minimize the following error, called the MSE.

$$\mathcal{L}_{MC} \doteq \|\boldsymbol{e}\|_D^2. \tag{6}$$

In expectation, Monte Carlo value learning minimizes 6, with errors estimated in an unbiased fashion from sampled episodes. In contrast, bootstrapping methods like Temporal Difference (TD) Learning (**?**) instead use the notion of the Bellman (or TD) error, given by:

$$\boldsymbol{\delta} = \boldsymbol{R} + \gamma P \boldsymbol{v} - \boldsymbol{v}. \tag{7}$$

TD learning is an iterative algorithm that is based on relationship between the Bellman error and the gradient of the value estimator. We will write its expected update in matrix form, stacking the gradients at each state into the $N \times k$ Jacobian matrix, J_θ:

$$J_\theta \doteq \begin{bmatrix} \nabla v_\theta(s_1)^\top \\ \nabla v_\theta(s_2)^\top \\ \vdots \\ \nabla v_\theta(s_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times k}. \tag{8}$$

In expectation, the TD update and loss are given by the following two equations.

$$\boldsymbol{\theta} \xleftarrow{TD} \boldsymbol{\theta} + \alpha J_\theta^\intercal D \boldsymbol{\delta} \tag{9}$$
$$\mathcal{L}_{TD} \doteq \|\boldsymbol{R} + P\overline{\boldsymbol{v}} - \boldsymbol{v}\|_D^2 \tag{10}$$

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Where $\overline{f_\theta}$ indicates the stop gradient operator is applied to θ. A *TD Fixed Point* is a value of θ where no update occurs in expectation. A TD fixed point is reached whenever $\mathbf{0}_k = J_\theta^\intercal D\boldsymbol{\delta} = \mathbb{E}_{s\sim\mu}(\delta(s)\nabla v(s))$.

It is known (**?**) that (9) can be rewritten using the substitution $A\boldsymbol{e} = D(I-\gamma P)(\boldsymbol{V}-\boldsymbol{v}) = D(\boldsymbol{R}-(I-\gamma P)\boldsymbol{v} = D\boldsymbol{\delta}$. This substitution is a key insight from (**?**)'s analysis of deep TD learning, and means (9) can be rewritten as:

$$\boldsymbol{\theta} \xleftarrow{TD} \boldsymbol{\theta} + \alpha J_\theta^\intercal A\boldsymbol{e} \tag{11}$$

3. Related Work

[TODO]

4. Policy Optimization Bellman-Smoothed Value Error

The Bellman-Smoothed value error from (**?**) is given by:

$$E \doteq \|\boldsymbol{V} - \boldsymbol{v}\|_S^2 = \boldsymbol{e}^\intercal A\boldsymbol{e} = \boldsymbol{e}^\intercal S\boldsymbol{e}. \tag{12}$$

Since S is positive definite, $E \geq 0$, with equality when the value error is zero at all states where $\boldsymbol{\mu}(s) > 0$. We study gradient descent on E, which makes the following update:

$$\boldsymbol{\theta} \xleftarrow{E} \boldsymbol{\theta} + \alpha J_\theta^\intercal S\boldsymbol{e} \tag{13}$$

Proposition 4.1. *The expected update due to gradient descent on E is equal to the expected update due to TD learning, less $J_\theta^\intercal K\boldsymbol{e}$, where $K = \frac{1}{2}(A - A^\intercal)$ is the skew-symmetric component of A.*

Proof.

$$J_\theta^\intercal S\boldsymbol{e} = J_\theta^\intercal(A-K)\boldsymbol{e} = J_\theta^\intercal A\boldsymbol{e} - J_\theta^\intercal K\boldsymbol{e} \tag{14}$$

□

Since the Bellman-Smoothed Value Error is the S-weighted norm of the value error vector, E can be treated as a squared error. It is convex in $\boldsymbol{v}$, and minimized when the value error is zero at all visited states. Under typical assumptions gradient descent can be shown to converge to a local minimum of E (as in supervised learning). In contrast, exact TD learning may not converge. Moreover, its fixed points may have positive value error (**?**). This is due to the skew-symmetric component K in the TD update. A TD fixed point can have non-zero value error if $J_\theta^\intercal S\boldsymbol{e} = -J_\theta^\intercal K\boldsymbol{e}$. The presence of K also implies that the TD-Learning update (11) can increase the value error E, possibly leading to divergence [TODO: prove]. (**?**) showed that when the Markov Chain is reversible ($DP = (DP)^\intercal$), TD learning is equivalent to gradient descent on E.[1] However, in practical RL, the MDP is rarely reversible.

Despite these theoretical limitations, on-policy TD learning is popular. For one, the sampled update (e.g. the Monte Carlo estimator of (9)) has low variance due to only incorporating one-step noise. Secondly, with tabular and linear function approximators, the TD-fixed point and the model-based solution are the same – a fact known as the certainty equivalence principle ((**?**), (**?**)). Third, TD-learning ascribes similar values to neighboring states, while Monte Carlo treats each state as disconnected.

As discussed later, E-minimization smooths the value function in a similar manner to TD learning, while remaining a safer optimization target. In the next section, we present empirical evidence that the exact version of E minimization compares favorably to exact TD and exact MC. Additionally, we present a form of E that can be sampled, with variance lying exactly between that of TD and MC.

Experiment: Comparing Expected Versions of the Algorithms We find empirically that TD learning performs worse than E minimzation when the Markov Chain has a significant skew-symmetric component. We study the behavior of E and $\mathcal{L}_{TD}$ and on nine environments, which are small enough to solve exactly.

- **Four Rooms (Sparse)**: 104 states (105 including terminal). A classic sparse reward navigation task on a 13×13 grid with 104 traversable cells partitioned into four rooms. The state representation is a grid, and a CNN is used for the value network.
- **Four Rooms + Mines (Sparse and Dense)**: Adds mines which terminate the agent's life. Dense versions of navigation tasks use potential-based reward shaping based on distance to the goal.
- **Eight Rooms (Sparse and Dense)**: 210 states (211 including terminal). An extended navigation task on a 25×13 grid with eight rooms.
- **Mountain Car (Sparse and Dense)**: A continuous state space (discreteized for tabular solvers). Dense rewards are obtained with potential (height)-based reward shaping.
- **Space Invaders**: 1,974 states (1,975 including terminal). A tabular 3-alien shielded-fleet version of Space Invaders on a 7×6 grid, with endless waves and termination only on death.
- **Whirlpool**: 105 states (105 including terminal). A gridworld using the same visual state representation as Four Rooms. The agent starts in the outermost part

[1] In the reversible case A is symmetric and $\boldsymbol{e}^\intercal K\boldsymbol{e}$ equals zero.

of the grid, and must navigate to the center of the whirlpool. A strong force pushes the agent clockwise, in the sense that the agent's action succeeds only 10% of the time, and 90% of the time the agent is pushed clockwise.

As we shall see, Whirlpool, Space Invaders, and Mountain Car all have strong skew-symmetry. Informally, these MDPs are far from reversible. A reversible MDP has the same probability flow transitioning "forward" from $s \to s'$ as it does transitioning "backward" from $s' \to s$. In skew-symmetric MDPs, either the dynamics move only forward, or a strong asymmetric force acts on the agent (e.g. gravity in Mountain Car).

The above environments are small enough that we can exactly compute the TD update (9), the MSE loss (6), and Bellman-Smoothed Value Error (12). This removes any noise in the updates. We use deep value function approximation, using a CNN on visual tasks and a MLP on Mountain Car. The only randomness across runs is from neural network initialization. Hyperparamters can be found in the Appendix [TODO: put in hypers]. We first consider fixed policy evaluation. For each algorithm, we train a neural network to estimate the value of the uniform random policy. We denote the size of the final hidden layer by k, and try $k = 8$ and $k = 64$ to understand the impact of the capacity of the network. Since the Markov Chain is fixed, we can compute its skew-symmetry as the ratio of $\|K\|$ to $\|S\|$. For a reversible Markov Chain, $\|K\|$ will be zero. We plot this skew-symmetry measure against the relative error – the average MSE during learning for E relative to TD. A lower relative error means E performed better. Note that we use the on-policy MSE, which is the typical objective in policy evaluation, though neither algorithm minimizes it directly.

Figure 1 shows that, as Proposition 4.1 suggests, when K is small compared to S, E minimization and TD have the same MSE during learning. This is because their learning dynamics are almost the same, as Proposition 4.1 suggests. However, as the environment becomes more skew-symmetric, TD learning's performance declines relative to E minimization.

While the MSE is the de-facto objective for fixed policy evaluation, the goal of policy evaluation is policy improvement. We run the three algorithms to evaluate the uniform random policy. In addition to TD and E-minimzation, we also consider **Exact Monte Carlo** (MC), which is the minimzation of $\mathcal{L}_{MC}$ directly by fitting the true value function in a supervised fashion. Despite the name "Monte Carlo", the exact MC update has no noise besides random neural network initialization.

Results are shown in 1. Exact MC usually obtains the best MSE by the end of learning, which makes sense as the MSE is its learning objective. We note that E-minimization learns the value function quickly – as measured by the training AUC of the MSE. We also consider two metrics designed to inform us of the quality of the learned value function for the purposes of policy improvement. Metric one is the start-state value of the greedy policy on the learned value function. This is the average return that would be obtained following the geredy policy. Metric one measures the strength of the policy due to a single round of approximate policy iteration. Metric two measures the quality of the policy gradient: the policy gradient can be shifted or scaled by a constant without affecting the policy update. As such, we measure the cosine-similarity, or normalized dot product, of estimated advantage and true advantage (the advantage is tread as a vector in state-action space). Since we learn value functions, we compute the estimated advantage as $\boldsymbol{a} = \boldsymbol{R} + \gamma P \boldsymbol{V}$. We observe that by both of these metrics, exact E-minimization learns a value function that is better for policy improvement.

Expected PPO. To see if these results carry over to policy optimization, we implement a no-sampling version of PPO, with a critic is due to exact E, exact TD, or exact MC. The policy is paramterized by an actor network with the same architecture as the critic, without parameter sharing between actor and critic. We use a constant learning rate for the actor, but tune the critic learning rate for each algorithm. Results for the best tuned setting are shown in Figure 2. Unlike the fixed policy evaluation task, PPO updates both the actor and critic at each round. This means that the learning algorithm must be able to evaluate a new policy quickly, with transfer between policies. We see that TD learning struggles with sparse rewards (see EightRooms Sparse and Dense), possibly because it would require more iterations to propagate value through the state space. However, TD also struggles with dense, skew-symmetric tasks like Dense Mountain Car and Space Invaders.

5. Sampling the Bellman-Smoothed Value Error

Theorem 5.1. *If $\boldsymbol{\mu}$ is the stationary distribution ($\boldsymbol{\mu}^\intercal P = \boldsymbol{\mu}$), the Bellman-Smoothed Value Error can be written as*

$$E = (1-\gamma)\|\boldsymbol{e}\|_D^2 + \gamma \mathcal{E}, \tag{15}$$

where the energy, $\mathcal{E}$, is given by the following.

$$\begin{aligned}\mathcal{E} &= \frac{1}{2}\sum_{ij} \mu_i p_{ij} (e_i - e_j)^2 \\ &= \frac{1}{2}\sum_{ij} \mu_i p_{ij} (\underbrace{r_i + \gamma v(s_j) - v(s_i)}_{=\delta_i} + (1-\gamma) e_j)^2 \end{aligned} \tag{16}$$

Equation 16 emphasizes that the energy $\mathcal{E}$ is the sum of squared terms, each of which can be estimated on-policy.

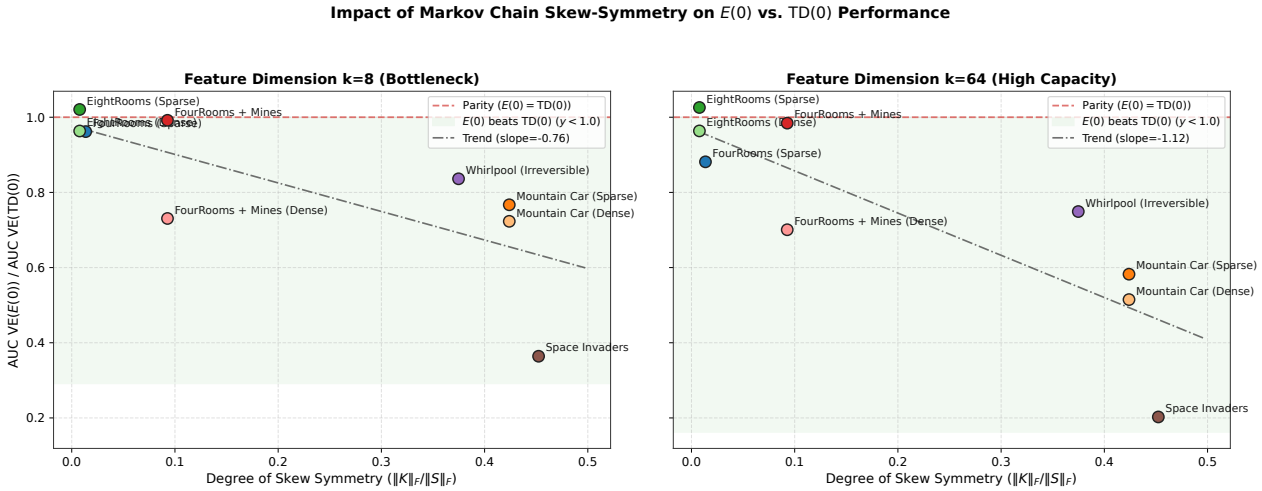


Figure 1. **Alignmnent vs. Skew Symmetry** Environments charted by their alignment based on the ratio of the norm of K to the norm of S and the relative performance of exact TD vs. exact E-Minimization $\frac{\text{AUC}(MSE_E)}{\text{AUC}(MSE_{TD})}$. The negative slope captures the fact that in more skew-symmetric environments, E has better value learning. 10 seeds.

Algorithm	MSE (AUC)	MSE (Final)	Advantage Acc.	Value of Greedy Policy
TD	(0.0/9) [0w+0t]	(0.3/9) [0w+1t]	(0.0/9) [0w+0t]	(2.0/9) [2w+0t]
MSE Minimization	**(4.5/9) [4w+1t]**	**(7.3/9) [7w+1t]**	(3.0/9) [2w+2t]	(3.0/9) [2w+2t]
E Minimization	**(4.5/9) [4w+1t]**	(1.3/9) [1w+1t]	**(6.0/9) [5w+2t]**	**(4.0/9) [3w+2t]**

Table 1. Win Rate of TD, MC, and E minimization across several metrics: The AUC and final MSE as well as two measures of policy improvement: (1) the cosine similarity between the learned and true advantage functions, and (2) the value of the greedy policy at the start staet. For each environment, an algorithm is a winner if it performs best on a certain metric. Ties occur when no single algorithm was statistically significantly best, and are split evenly. Thus $1.5/9$ indicates that the algorithm won outright on exactly one environment and tied on another.

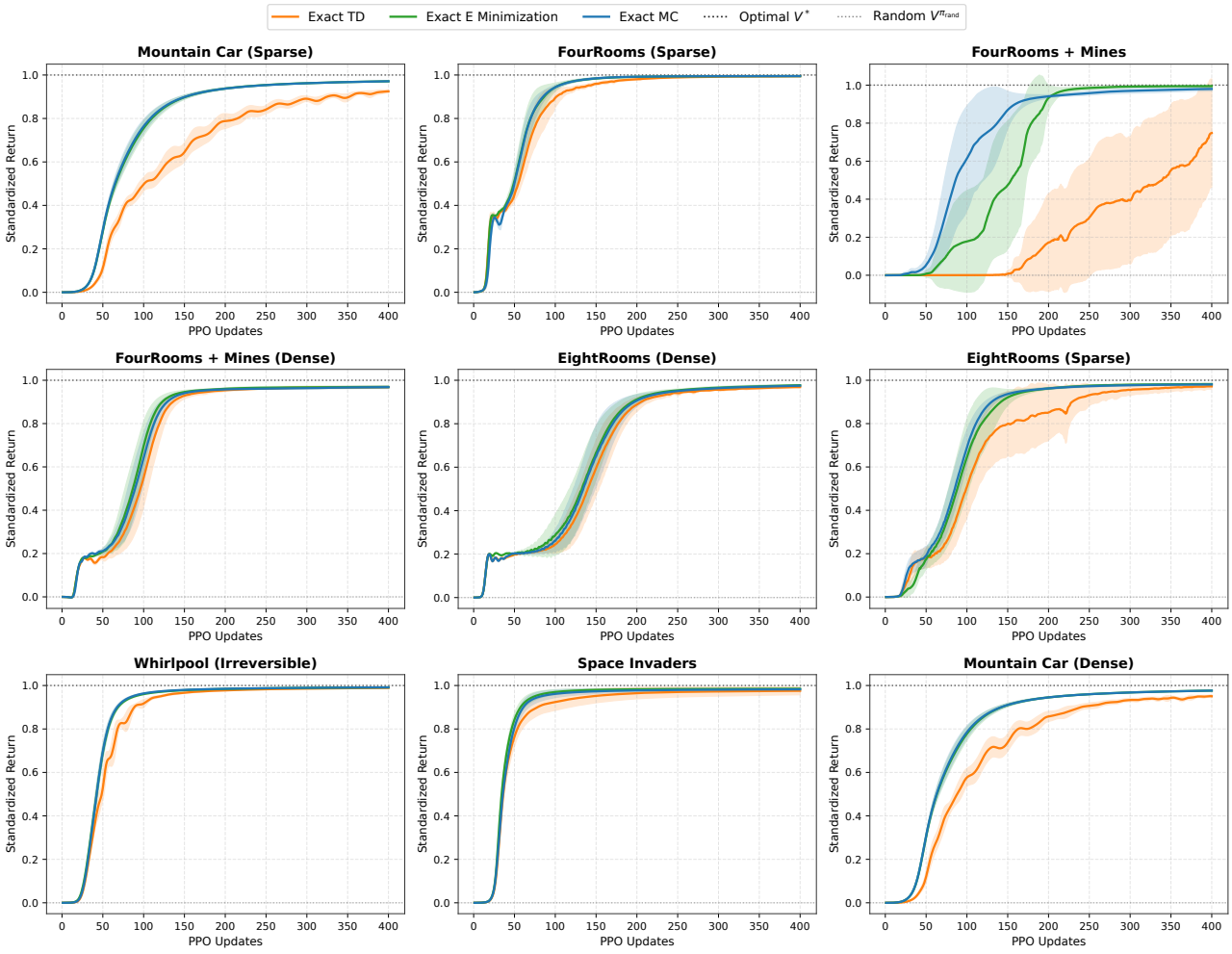


Figure 2. **Exact PPO.** Learning curves for no-sampling PPO with the critic due to exact TD, MC, or E-minimization. 10 seeds.

Each squared term is the sum of the sampled TD error and *next state* Monte-Carlo error. Notice that the total weight on Monte-Carlo terms is $O((1-\gamma)^{-1})$, reducing their variance.

Proposition 5.2. *The Bellman-Smoothed Value Error is equal to the D-weighted inner product of the value error and the TD-error.*

$$E = \boldsymbol{e}^\intercal D \boldsymbol{\delta} \tag{17}$$

Corollary 5.3. *The magnitude of E is bounded by the magnitude of the Monte-Carlo and TD losses.*

$$E = \langle \boldsymbol{e}, \boldsymbol{\delta} \rangle_D \leq \|\boldsymbol{e}\|_D \|\boldsymbol{\delta}\|_D \tag{18}$$

One can obtain an unbiased sample of e_i using the empirical return $G_i \doteq \sum_{t \geq 0} \gamma^t r_t$.

Gradient of the Energy Recall that $\boldsymbol{e}$ depends on θ, with $\nabla_\theta \boldsymbol{e} = -\nabla_\theta v_\theta$. The gradient of the energy can be written as the sum of many terms, weighted by the frequency under $\boldsymbol{\mu}$:

$$\nabla_\theta \mathcal{E} = \mu_i \underbrace{\nabla_\theta \frac{1}{2} \sum_i \sum_j p_{ij} (e_i - e_j)^2}_{\nabla_\theta \mathcal{E}_i}$$

Further examining each of these terms:

$$\begin{aligned}
\nabla_\theta \mathcal{E}_i &= \frac{1}{2} \sum_j p_{ij} \nabla_\theta (e_i - e_j)^2 \\
&= \sum_j p_{ij} (e_i - e_j) \nabla_\theta (e_i - e_j) \\
&= \sum_j p_{ij} (e_i - e_j) (\nabla_\theta v_\theta(s_j) - \nabla_\theta v_\theta(s_i)) \\
&= \underbrace{\sum_j p_{ij} (e_i - e_j) \nabla_\theta v_\theta(s_j)}_{\text{term (1)}} \\
&\quad \underbrace{- \sum_j p_{ij} (e_i - e_j) \nabla_\theta v_\theta(s_i)}_{\text{term (2)}} \qquad (19) \\
&= \mathbb{E}_{j \sim P_i} \big[\underbrace{(e_i - e_j) \nabla_\theta v_\theta(s_j)}_{\text{term (1)}} - \underbrace{(e_i - e_j) \nabla_\theta v_\theta(s_i)}_{\text{term (2)}} \big] \qquad (20)
\end{aligned}$$

This decomposition splits the gradient into terms depending on the current state (Term (2)), and those depending on **expected** gradient at the next state (Term (1)). We call Term (2) the *forward term*. It can be rewritten using the identity $\boldsymbol{\delta} = \boldsymbol{e} - \gamma P \boldsymbol{e}$ as:

$$\text{Term (2)} = \nabla_\theta v_\theta(s_i) [\delta_i - (1-\gamma) \sum_j p_{ij} e_j]. \tag{21}$$

Note that Term (2) resembles TD learning plus another Monte-Carlo like term. In matrix form, it is written as

$$-J_\theta^\intercal D (I - P) \boldsymbol{e} = -J_\theta^\intercal D \boldsymbol{\delta} + (1-\gamma) J_\theta^\intercal D P \boldsymbol{e} \tag{22}$$

The cross term adjusts the prediction at s_i so that its error matches the expected future error $P\boldsymbol{e}$. Term (2) can be estimated with $\hat{e}_t = G_t - v_t$ and the empirical Bellman error $\hat{\delta}_t$, with no bias.

Biased sampling of the next-state term We call Term (1) the *backward term*, since the value gradient is evaluated at the next state's predicted value. Estimating e_i with Monte Carlo sampling obtains a biased estimate of Term (1). To see this, we decompose it as follows:

$$\text{Term (1)} = \underbrace{e_i \sum_j p_{ij} \nabla_\theta v_\theta(s_j)}_{\text{biased}} - \sum_j p_{ij} e_j \nabla_\theta v_\theta(s_j) \tag{23}$$

The issue is that the empirical (Monte Carlo) error $\hat{e}_i = G_i - v_\theta(s_i)$ depends on the sample of $s_j \sim P(s_j|s_i)$. In expectation, this gives $e_i = \mathbb{E}_{G_j}[r_i + \gamma G_j - v_\theta(s_i) \mid s_i]$. Thus, the first term in Equation 23 is a product of expectations $\mathbb{E}_{G_j}[\hat{e}_i]\mathbb{E}_{s_j}[\nabla_\theta v_\theta(s_j)] \neq e_i \mathbb{E}_{s_j}[\nabla_\theta v_\theta(s_j)]$. This is reminiscent of the classic Double Sampling problem due to Bellman Error Minimization, where the product of expectations $\mathbb{E}_{s_j}[v_\theta(s_j)]\mathbb{E}_{s_j}[\nabla v_\theta(s_j)]$ appears.

Resolving the Double Sampling Issue To resolve this without requiring two independent transitions from s_i, we introduce an auxiliary value network v_{aux} trained on T-step Monte Carlo returns G_i^T to estimate V, and define the proxy error $e_{\text{aux}}(s) \doteq v_{\text{aux}}(s) - v_\theta(s)$. We use v_{aux} in **Term (1) only**, continuing to use $\hat{e}_i$ in all other terms. The final gradient estimate we use is:

$$\begin{aligned}
\hat{g}_i = \quad & \underbrace{-2(1-\gamma) \hat{e}_i \nabla_\theta v_i}_{\text{weighted MC}} \\
+ \quad & \underbrace{\gamma \big(e_{\text{aux},i} - e_{\text{aux},j} \big) \nabla_\theta v_j}_{\text{Term (1)}} \\
- \quad & \underbrace{\gamma \big(\hat{e}_i - \hat{e}_j \big) \nabla_\theta v_i}_{\text{Term (2)}}, \qquad (24)
\end{aligned}$$

This gradient corresponds to the following loss:

$$\boxed{l_i(\theta) = (1-\gamma)\hat{e}_i^2 + \frac{\gamma}{2} \Big(\hat{e}_i - \text{sg}(\hat{e}_j) \Big)^2 + \frac{\gamma}{2} \Big(\text{sg}(e_{\text{aux},i}) - e_{\text{aux},j} \Big)^2}$$

Note that without Term (1), this gradient would be due to a mix of Monte Carlo and TD learning. We test this as

an ablation, calling it *forward only* since it zeros out the backward term.

$$\hat{g}_{\text{forward-only},i} = -\big[2(1-\gamma)\hat{e}_i + \gamma\big(\hat{\delta}_i - (1-\gamma)\hat{e}_j\big)\big]\nabla_\theta v_i$$

Results We call the variant that uses an auxiliary value network v_{aux}. We test it and the forward only variant against TD(0) and MC, as well as a naive sampling variant ("Oct 8"). We first show the value error (Monte Carlo Objective) due to a fixed policy evaluation task in Figure 3. We observe that in all environments, Forward Only and v_{aux} perform better than naive sampling. We also notice that on Mountain Car, the forward only variant does worse than using an auxiliary network.

Next, we test these algorithms for control, using them to train the critic in PPO. Observe that Oct 8 does poorly due to the bias in the gradient (although perhaps not as poorly as one might expect given that its value error is an order of magnitude higher).

Finally, we examine the alignment of the sampled gradient to the ground truth gradient for a realistic setting: using policies obtained by a seperate policy optimization algorithm, and batch rollouts as in PPO. Figure 5 plots the results during learning. We measure gradient alignment with the cosine similarity $\rho(\hat{g}, g)$ where $g = \nabla E$, and $\hat{g}$ is the estimate. Somewhat striking is that the Forward Only variant, which drops the backward term, actually has strong gradient alignment on Space Invaders. Despite this environment having strong asymmetry, we speculate that the directed nature of the task is responsible for the forward and backward term being strongly similar. Notably, only the version that uses an auxiliary value network maintains positive alignment throughout learning. The forward only variant is misaligned due to bias on the maze tasks, and the double-sampling version is almost always worse than the version that uses an auxiliary network.

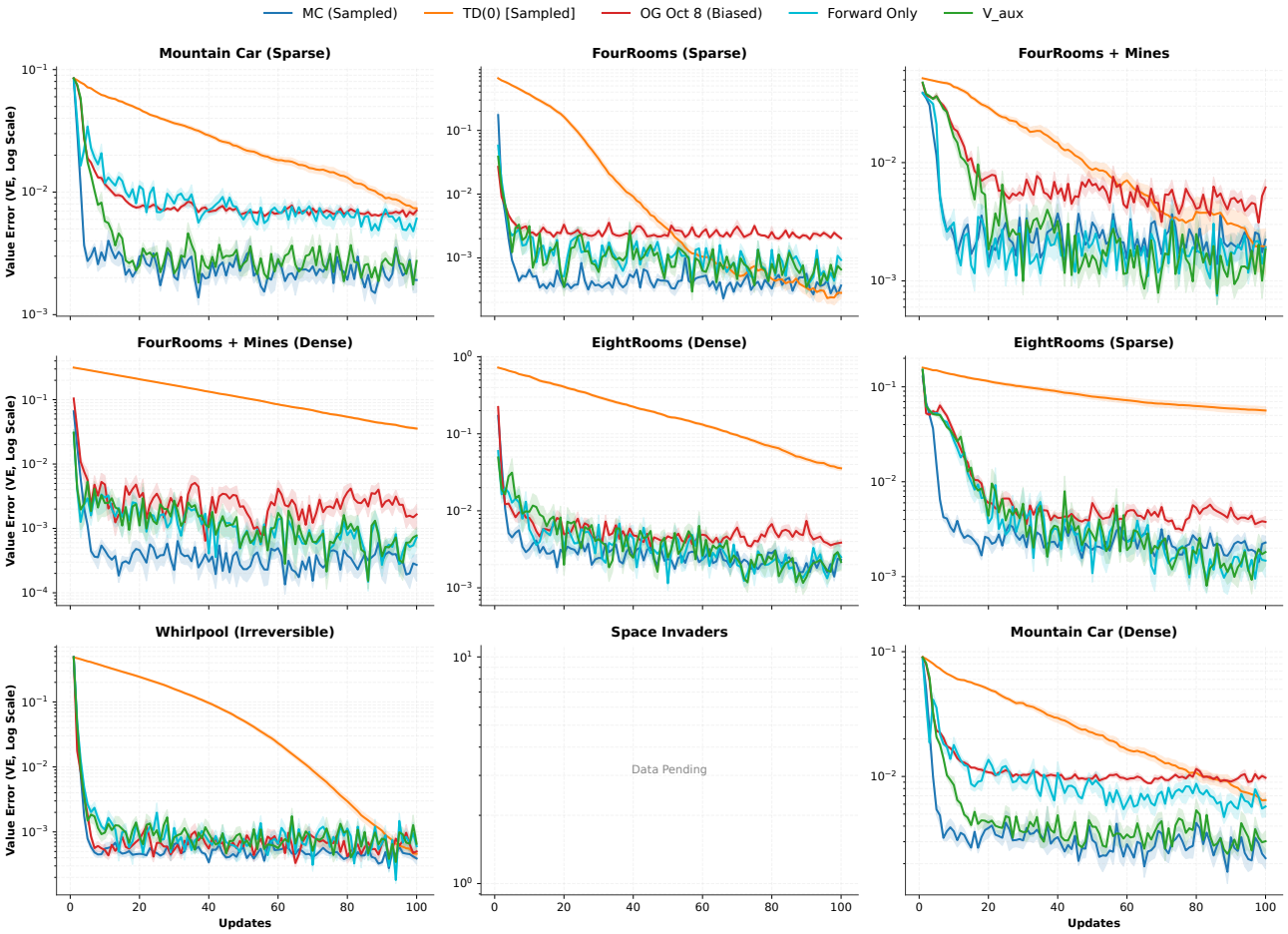


Figure 3. **Value Error in Fixed Policy Evaluation** 10 seeds.

6. $E(\lambda)$

The TD(λ) algorithm (**?**) interpolates between one-step-bootstrapping ($\lambda = 0$) and Monte Carlo value learning ($\lambda \to 1$). The Bellman-Smoothed value error be extended naturally in the same way:

Definition 1. The λ-Bellman Smoothed value error generalizes E, and is defined as follows.

$$E(\lambda) \doteq \frac{1}{2}\boldsymbol{e}^\intercal A(I-\gamma\lambda)^{-1}\boldsymbol{e} \tag{25}$$

Theorem 6.1. *In expectation, when sampling states from μ, the TD(λ) update is:*

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{1}{2}\alpha\left[\nabla_\theta E(\lambda) - J_\theta^\intercal K_\lambda e\right] \tag{26}$$

Where K_λ is the skew-symmetric component of $A(I-\gamma\lambda)^{-1}$.

We can obtain a sampling form of $E(\lambda)$ by defining a row-stochastic transition matrix $P_\lambda \doteq (I-\gamma\lambda)P(I-\gamma\lambda)^{-1}$. Since P_λ and P have the same stationary distribution, we can write:

Theorem 6.2. *Define an effective discount rate $\tilde{\gamma} = \frac{\gamma(1-\lambda)}{1-\gamma\lambda}$. Note that $\lim_{\lambda\to 1}\tilde{\gamma} = 0$. If $\boldsymbol{\mu}$ is the stationary distribution, then $E(\lambda)$ can be written as:*

$$E(\lambda) = (1-\tilde{\gamma})\|\boldsymbol{e}\|_D^2 + \frac{\tilde{\gamma}}{2}\sum_{i,j}\mu_i[P_\lambda]_{ij}(e_i-e_j)^2 \tag{27}$$

Taken together, theorems 6.1 and 6.2 emphasize that increasing λ has two effects:

1. **Anchor weighting:** Increases the weight on the MSE term $(1-\tilde{\gamma})\|\boldsymbol{e}\|_D^2$.
2. **Kernel reach:** Increases the "smoothing radius," or how far along a trajectory the energy term smooths.

6.1. Sampling $E(\lambda)$

We consider sampling $E(\lambda)$ using an auxiliary value network – just as we did in the prior section. To sample from P_λ, we use the fact that P_λ is a geometric mixture of multi-step distributions. Writing P_λ in the following

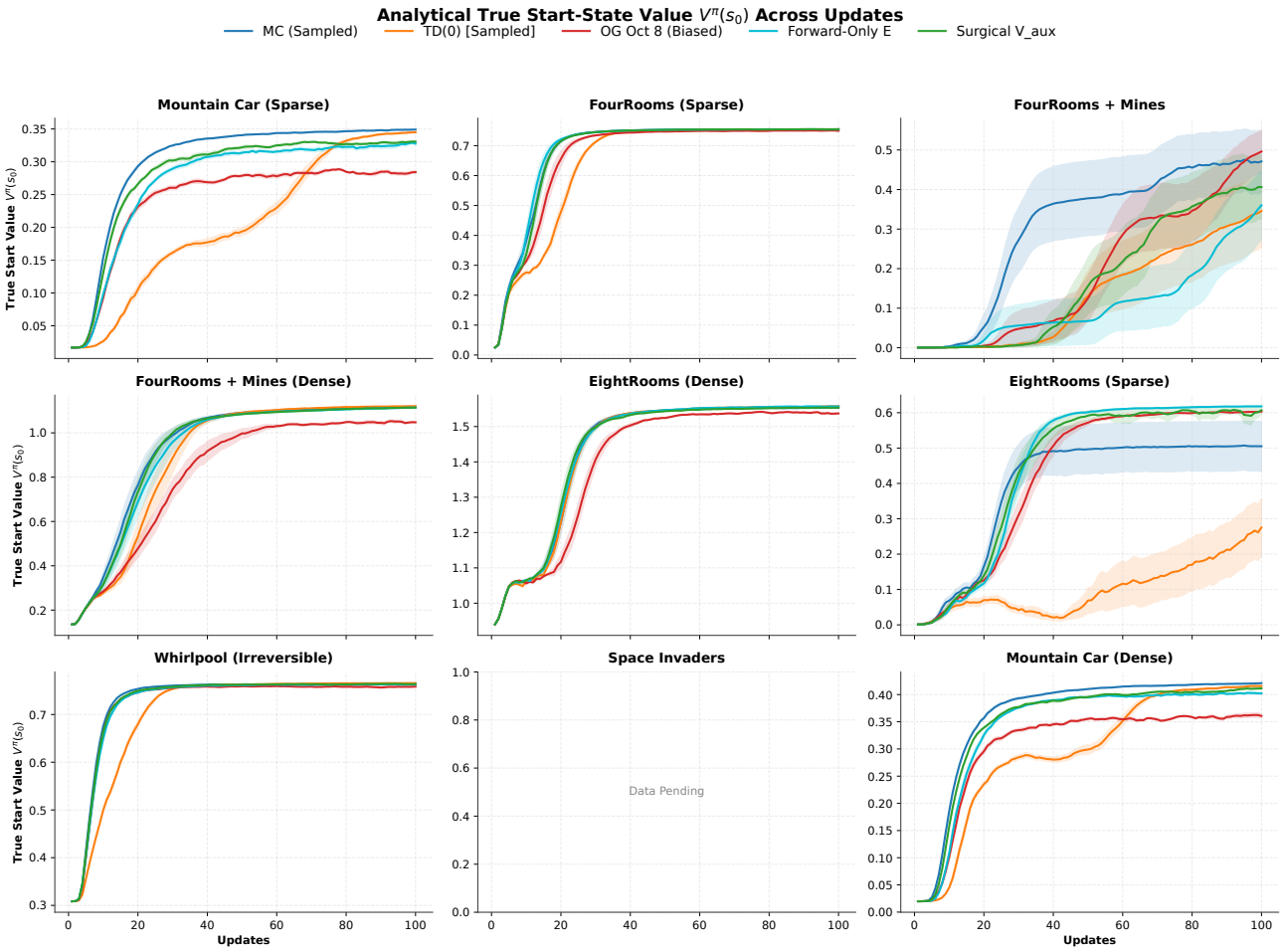


Figure 4. **PPO Performance**: The ground-truth value of the start state is shown, eliminating variance due to measuring returns. 10 seeds.

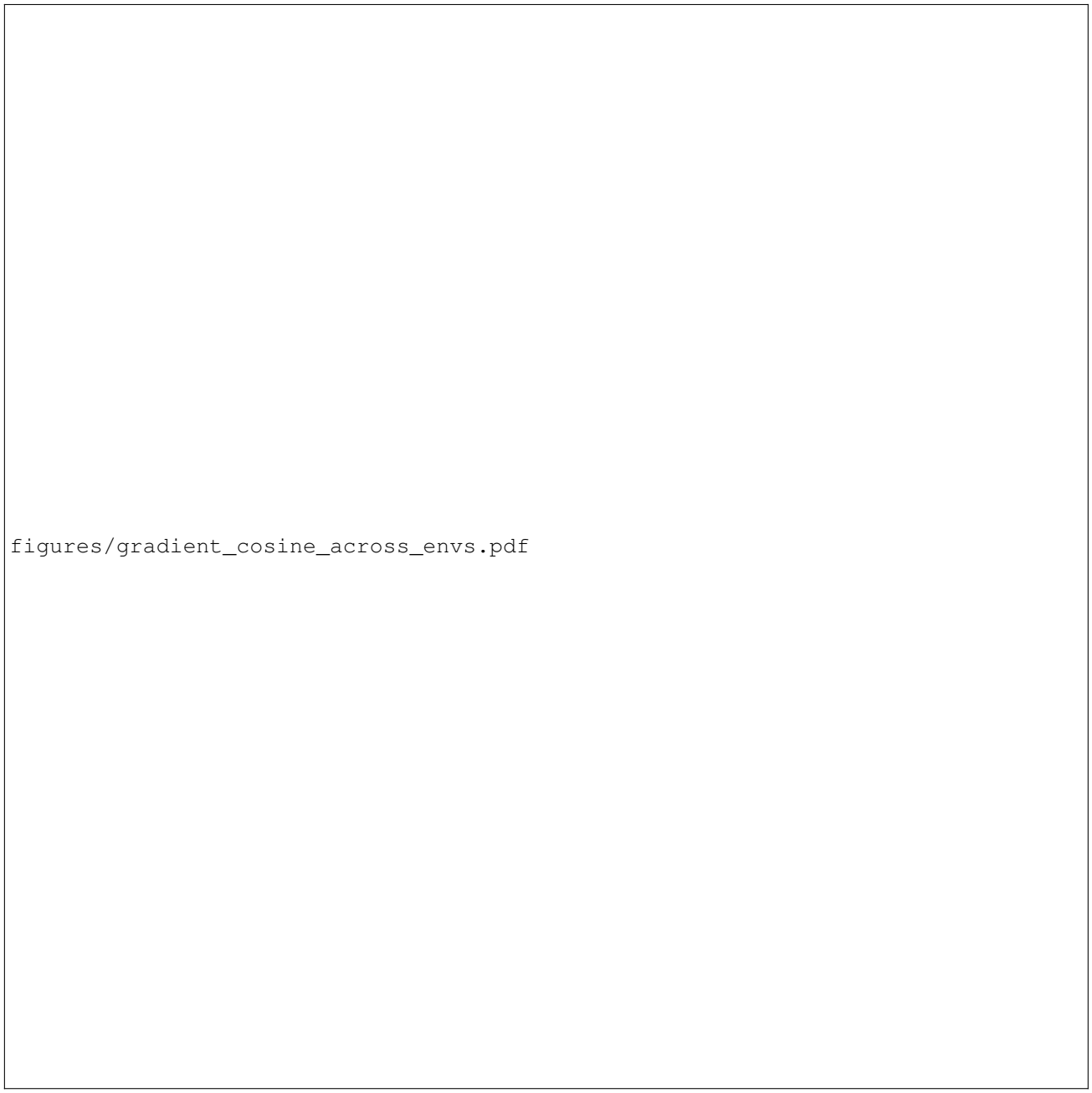


Figure 5. **Gradient Accuracy**: We compare the ground truth gradient of E with three sampled variants: October 8, which uses raw returns and suffers from double sampling; Forward Only which drops the backward term, incurring bias; and V Aux, which uses an auxiliary network in the backward term. The cosine similarity varies between 1 (directional alignment), to -1, full misalignment, and is 0 when the gradients are orthogonal.

form shows that it weights each m-step transition matrix P^m with weight $(1-\gamma\lambda_(\gamma\lambda)^m$. This corresponds to the expected value of P^m where the jump length m is due to a geometric distribution $\mathbb{E}[m] = \frac{1}{1-\gamma\lambda}$. This mirros the alternative view of discounting based a modified undiscounted MDP that terminates with probability $1-\gamma$ every step.

$$P_\lambda = \sum_{m=1}^{\infty} (1-\gamma\lambda)(\gamma\lambda)^{m-1} P^m.$$

Based on this, a Monte Carlo estimator for $\mathbb{E}_{i\sim\mu,\, j\sim P_\lambda(i,\cdot)}\left[(e_i - e_j)^2\right]$ can be obtained by sampling state $i \sim \boldsymbol{\mu}$ and jump length $m \sim \text{Geom}(1-\gamma\lambda)$, letting j be the state reached after m steps. This allows one to sample $E(\lambda)$ from on-policy rollouts, maintaining backward differentiability through $v(s_j)$. This geometric jump perspective, where $\mathbb{E}[m] = \frac{1}{1-\lambda\gamma}$, tangibly demonstrates how the smoothing radius grows with λ.

Experiment: Sampled $E(\lambda)$ Gradient Alignment. (Tables 2 and 3) In practice, we collect fixed-length rollouts – typically of $T = 128$ or $T = 256$ steps. This presents an issue when the jump length exceeds the roll-out length, something that becomes more common at higher values of λ. We considered two ways to handle this: (1) zeroing out the critic update for timestep t, and (2) zeroing out the smoothing term, and updating based on the empirical return only. We compare the ability of each of these sampling methods to estimate the ground truth gradient in Table 2. Using the Monte Carlo term adds a small amount of bias, but maintains a strong cosine similarity and lowers the variance of the gradient estimator. Per-Environment analysis reveals that Space Invaders, an infinite horizon task that is approximately a DAG, is entirely responsible for the strong results due to Forward Only variant, but that across environments, the gradients due to the v_{aux} methods are more aligned with the analytical gradient in weight-space.

Experiment: $E(\lambda)$ vs Bellman Smoothed Monte Carlo (Figure 6) As mentioned earlier, $E(\lambda)$ differs from $E(0)$ in exactly two ways: (1) higher anchor weight, and (2), longer jumps. To disentangle these effects, we consider Bellman Smoothed Monte Carlo (BSMC), which engages in one-step smoothing of the Monte Carlo loss only. Precisely, it has objective $\mathcal{L}_{bsmc} = w_{mc}\|\boldsymbol{e}(\theta)\|_D^2 + (1-w_{mc})\mathcal{E}(\theta)$. We compare BSMC to $E(\lambda)$ by setting $w_{\text{MC}}(\lambda) = \frac{1-\gamma}{1-\gamma\lambda}$ to match $E(\lambda)$. Results are shown in Figure 6. At intermediate values of λ, $E(\lambda)$ outperforms BSMC, implying that the m-step smoothing is beneficial on our testing suite of environments. However, at high values of λ, BSMC outperforms. We can say that $E(\lambda)'s$ failure to match BSMC at high λ is due to the jump length m. However, whether this is because the variance in $m \sim \text{Geom}(1-\gamma\lambda)$ or because $\mathbb{E}[m]$ grew too large to be helpful for our test suite ($\mathbb{E}[m] \approx 17$ for $\lambda = 0.95$) cannot be ascertained from these experiments alone.

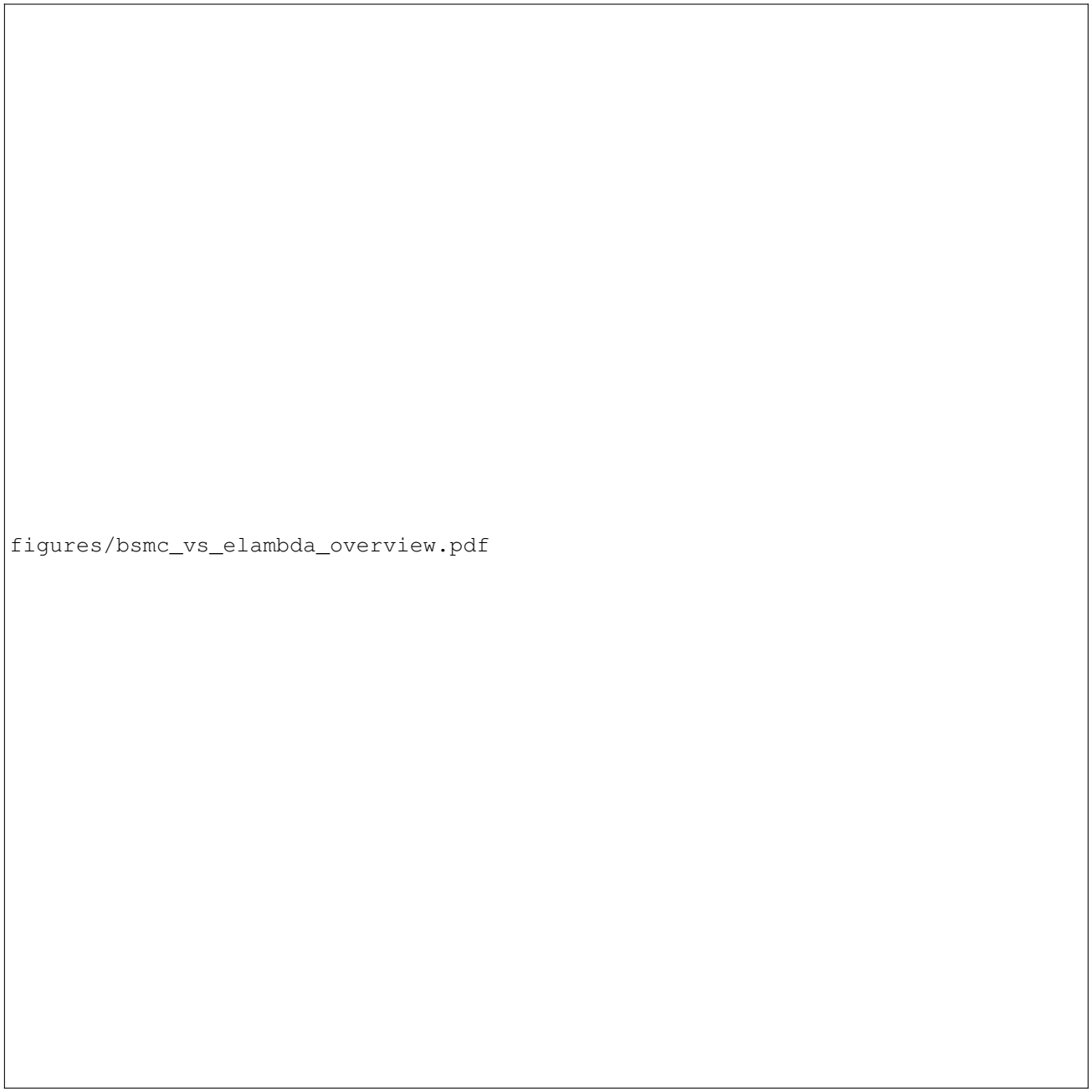


Figure 6. **Comparison to BSCM**: We compare runs of PPO where the critic is due either to BSMC or $E(\lambda)$ at various weightings of the Monte Carlo term. Per 10 seeds.

Table 2. Aggregated Sampled $E(\lambda)$ Performance Summary across 8 Benchmark Environments ($\lambda = 0.8, \gamma = 0.99$).

Formulation	**Cosine Sim** $\cos(\bar{g}, g^*) \uparrow$	**Variance** $\mathrm{Var}(\bar{g}) \downarrow$	**MSE** $\|\|\bar{g} - g^*\|\|^2 \downarrow$	**MSE vs Naive**
Naive (Oct 8 Analog)	+0.4101	4.68e-03	1.4590	1.00×
V_{aux} (Drop Invalid Jump)	+0.5559	1.23e-02	0.2252	0.15×
V_{aux} (MC Invalid Jump)	+0.5558	1.14e-02	0.2335	0.16×
Forward Only	+0.3187	7.26e-03	0.0182	0.01×

Table 3. Empirical comparison of Sampled $E(\lambda)$ gradient formulations against Exact Matrix $\nabla E(\lambda)$ across 8 benchmark environments ($\lambda = 0.8, \gamma = 0.99$). Cosine similarity measures directional alignment (↑), while Variance and MSE measure estimator dispersion and accuracy (↓).

	Cosine Sim ↑				**Variance ↓**				**MSE ↓**			
Environment	Naive	Drop	MC	Fwd	Naive	Drop	MC	Fwd	Naive	Drop	MC	Fwd
fourrooms-dense	**0.60**	**0.60**	**0.60**	-0.07	**2.0e-4**	5.4e-4	5.0e-4	3.8e-4	2.6e-3	1.8e-3	1.7e-3	**9.5e-4**
FourRooms-misc	0.47	**0.55**	0.54	-0.07	8.8e-5	1.8e-4	1.6e-4	**7.8e-5**	2.2e-3	1.8e-3	**1.7e-3**	2.0e-3
fourrooms-mines	0.24	0.01	0.01	**0.30**	2.6e-5	3.4e-5	3.1e-5	**1.8e-5**	**4.3e-5**	2.4e-4	2.2e-4	2.5e-4
fourrooms-mines-dense	-0.28	**0.50**	**0.50**	0.11	5.4e-4	5.7e-4	5.3e-4	**4.1e-4**	7.0e-3	6.5e-4	**6.1e-4**	8.0e-4
eightrooms-dense	**0.55**	0.53	0.53	0.54	**2.3e-4**	6.1e-4	5.7e-4	6.1e-4	3.1e-3	**1.2e-3**	**1.2e-3**	1.7e-3
eightrooms-misc	0.49	**0.58**	**0.58**	0.18	5.1e-5	7.7e-5	7.2e-5	**4.3e-5**	7.4e-4	6.0e-4	5.6e-4	**4.6e-4**
SpaceInvaders	0.46	0.85	0.85	**0.93**	**0.04**	0.10	0.09	0.06	11.66	1.80	1.86	**0.14**
mountaincar-dense	0.75	**0.83**	**0.83**	0.64	**3.9e-6**	6.6e-6	6.2e-6	**3.9e-6**	**1.3e-5**	1.8e-5	1.6e-5	1.7e-5
Mean (All Envs)	0.41	**0.56**	**0.56**	0.32	**4.7e-3**	0.01	0.01	7.3e-3	1.46	0.23	0.23	**0.02**